

DATA 101: Final Exam

Deadline: May 7 at 5pm | Email delivery with subject line: "Name, Data 101 Final"

Instructions. Answer all questions in a Jupyter notebook and turn in your assignment to me via email. Your email should include both the Jupyter notebook file (.ipynb) and the HTML output file exported from the notebook. Write clearly and show your reasoning where applicable using a combination of Markdown text and code inside Jupyter. You may use any material from your notes or the internet, as long as you understand and can explain all parts of the code you submit. Important: Use only the libraries we have learned in class. You will lose points if you use libraries or methods we have not learned in class.

1. (30 points) For this problem you will use two data sources. First is a book of your choice from Project Gutenberg's top 100 downloaded books which you can [access here](#). The second source is an online repository with famous paintings from: https://en.wikipedia.org/wiki/Wikipedia:Featured_pictures/Artwork/Paintings.

Here is your task: Match a book from Project Gutenberg with a painting from the repository above. Explain in depth why you chose this particular painting to match with the book you chose. Use arguments backed by data. Your answer should include a minimum of 250 words and 3 figures you create that best support your argument. Include all your work: text, code, figures in your answer.

2. (40 points) Read the following data set that contains newspaper articles into Python and name the dataframe as `news`. The data is available [here](#).

Here is your task: Test whether Zipf's Law holds in this newspaper data. Combine all article contents into one corpus. Clean the text by making all words lowercase and removing punctuation. Do not remove stopwords for this problem. Create a frequency table that has one row per word and includes the word's count and rank, where the most common word has rank 1, the second most common word has rank 2, and so on.

Make a figure that shows the relationship between word rank and word frequency. Based on your figure, does Zipf's Law seem to hold in this newspaper corpus? Explain your answer clearly. Where does the pattern fit well? Where does it break down? If the top-ranked word appears k times, Zipf's Law predicts that the word ranked 20th should appear about $k/20$ times. Compare this predicted count to the actual count of the 20th-ranked word in your frequency table. Is the prediction close?

Next, repeat the same rank-frequency analysis for just one article from the dataset. Does Zipf's Law seem to hold better for the full newspaper corpus or for one article? Why do you think that is? Finally, choose one word from your full-corpus frequency table and inspect a few examples of how that word is used in the articles. What is the word's rank, and does its rank make sense after you inspect how it appears in context? Explain.

Hint. To create one large corpus, you can combine the article texts into one long string. For example, if the article text column is called `bodyContent`, you can use:

```
" ".join(news["bodyContent"].dropna())
```

3. (30 points) For this problem, choose one book from Project Gutenberg's top 100 downloaded books and download its plain text file. You can [access the books here](#).
- (a) Read the book into Python as a text file. Clean the text by making all words lowercase and removing punctuation.
 - (b) Unzip the world map file posted in the same folder as this exam. Load the shapefile using GeoPandas and call the dataframe `gdf`. Use the country names in the map's `ADMIN` column as your list of country names.
 - (c) Count how many times any country is mentioned in the book. Create a dataframe called `country_mentions` with one row per country and a column called `mentions`. Reminder: the `.str.contains()` function can help you check whether a text column contains a word or phrase.
 - (d) Merge `country_mentions` with the GeoPandas world map dataframe, `gdf`.
 - (e) Make a static map using Matplotlib where countries mentioned more often in the book appear darker.
 - (f) Interpret the map you produced. Which countries are mentioned more often? Does this geographic pattern make sense given the book's setting, plot, or historical context? Explain at least two limitations of counting country names this way.

Hints. The world map file `ne_110m_admin_0_countries.zip` is posted in the same folder as this exam. Unzip all contents of the zip file and read in the shapefile with:

```
gdf = gpd.read_file("path_to_your_shapefile.shp")
```

One way to keep tally of country mentions is to first create a dataframe from the country names in `gdf["ADMIN"]`. Then create a new column called `mentions` that is initially set to 0. After that, loop over the countries and update the `mentions` column.

```
country_mentions = pd.DataFrame(gdf["ADMIN"])
country_mentions["mentions"] = 0 # create mentions col set at 0 initially
```

You can merge the country-count dataframe with the map using:

```
combined = gdf.merge(country_mentions, on="ADMIN", how="left")
```

After merging, countries that were never mentioned may have missing values in the `mentions` column. You can replace those missing values with 0:

```
combined["mentions"] = combined["mentions"].fillna(0)
```

To make countries with more mentions appear darker, use:

```
combined.plot(column="mentions")
```

Country names in text are messy. For example, `United States of America` might appear as `United States`, `America`, or `U.S.`; `United Kingdom` might appear as `England`, `Britain`, or `Great Britain`; `Russia` may appear where the map says `Russia`, but older books may say `Russian Empire`; and country names such as `Czechia` or `Myanmar` may not match older texts. You do not need to handle every special case, but you should take care of the main, easy cases that matter for the book you chose and explain your choices.